

Anomaly Detection in Industrial Equipment Using Isolation Forest

Moye Nyuysoni Glein Perry
Machine Learning
Hochschule Hamm Lippstadt
Email: moye-nyuysoni.glein-perry@stud.hshl.de

Abstract—Early fault detection is essential for reducing downtime and maintenance costs in industrial systems. This study applies the unsupervised Isolation Forest (IF) algorithm to a real-world dataset of 7,672 multivariate sensor readings from turbines, compressors, and pumps. We use grid search to optimize key hyperparameters and evaluate performance using precision, recall, and F1-score to address class imbalance. The tuned model achieves a test precision of 0.878, recall of 0.895, and F1-score of 0.887, effectively identifying rare faults with minimal false alarms. These results demonstrate the suitability of IF for scalable, interpretable anomaly detection in predictive maintenance contexts.

KEYWORDS

Isolation forest(IF) , True Positives (TP),False Positives (FP),False Negatives (FN) ,True Negatives (TN), Equipment Monitoring, Fault Detection, Sensor Data, Anomaly Detection, Compressor, Data Visualization.

I. INTRODUCTION

Industrial equipment monitoring is critical for preventing costly failures and optimizing maintenance schedules. Modern machinery, such as turbines and compressors, generates multivariate sensor data (e.g., temperature, pressure, vibration) that can reveal early signs of degradation or faults. Traditional threshold-based methods often fail to capture complex, non-linear relationships in such data, while supervised learning requires extensive labeled anomalies—a rarity in real-world scenarios.[2]

Isolation Forest (IF) addresses these challenges by exploiting the inherent sparsity of anomalies in feature space. Unlike density- or distance-based methods, IF isolates anomalies through random recursive partitioning, achieving linear time complexity and robustness to high-dimensional data. This study adapts IF to a real industrial dataset from Kaggle, which includes labeled sensor readings from geographically distributed equipment (e.g., pumps in Chicago, turbines in San Francisco).[3]

The contributions of this paper are:

- 1) Adapting Isolation Forest to detect anomalies in electronic signal measurements.
- 2) Utilizing a synthetic dataset from Kaggle to simulate sensor behavior and injected faults.
- 3) Providing visualization of anomaly isolation in signal space.

The remainder of this paper is structured as follows. An explanation of how the Isolation Forest algorithm functions is provided in Section II. Section III describes the sensor dataset. Section IV outlines the Isolation Forest implementation. Section V presents experimental findings. Section VI offers insights, and Section VII concludes and proposes future work.

II. ISOLATION FOREST

The Isolation Forest (iForest) is an anomaly detection algorithm that works by isolating data points instead of profiling normal behavior, which makes it especially efficient for detecting outliers in high-dimensional data. [7]

The core idea is simple: anomalies are easier to isolate than normal data. In other words, since anomalies are rare and different from the rest of the data, they can usually be separated from other points with fewer steps

A. How Isolation Happens

To isolate a data point, the algorithm randomly picks a feature (a variable or attribute in the dataset), and then randomly selects a value between the minimum and maximum of that feature to split the data. This process repeats recursively, creating a tree structure where each data point eventually ends up in its own branch (or “leaf”).

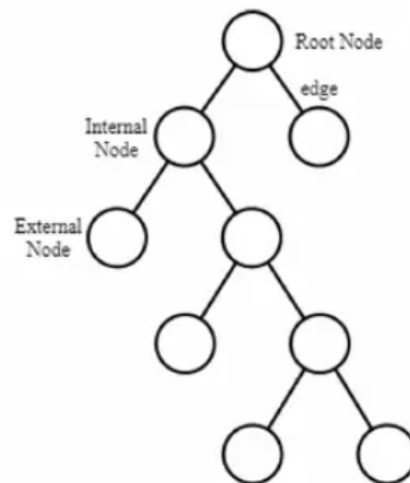


Fig. 1. Itree Structure
[1]

As observed in Figure 1, each tree in the Isolation Forest can contain two types of nodes: internal nodes, which have exactly two children, and external nodes, which have no children and represent isolated samples. A sample is considered isolated when it reaches an external node through recursive splits. The number of such splits (i.e., the number of edges traversed from the root node to the external node) defines the path length. Anomalies, by nature, tend to be isolated closer to the root, resulting in shorter path lengths, whereas normal data points typically require more partitions and therefore have longer paths[1]

This approach is much faster than traditional methods, especially on large datasets, because it doesn't rely on distance calculations or probability estimates.

B. Anomaly Score

Each point is given an anomaly score based on the average number of steps (path length) required to isolate it across a group (or "forest") of randomly constructed trees. A shorter average path means the point is more likely to be an outlier[1][7]

C. Contamination Parameter

iForest includes a contamination parameter, which is used to estimate the proportion of anomalies in the dataset. This helps the algorithm determine a threshold for the anomaly score, classifying points above that threshold as outliers [1]

Performance Evaluation Metrics

To evaluate the performance of the anomaly detection model, several standard classification metrics were used, including accuracy, precision, recall, and F1-score. These metrics are defined based on the confusion matrix terms: true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN).

Accuracy measures the overall correctness of the classifier:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision quantifies the proportion of correctly identified anomalies out of all instances classified as anomalies:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall (also known as sensitivity) indicates how many actual anomalies were correctly detected:

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score provides a harmonic mean of precision and recall, giving a balanced view of the model's performance:

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

[5]

D. Summary of Key Concepts

The Isolation Forest algorithm works by isolating data points through recursive splitting until each point is separated. It calculates an anomaly score based on how easily a point can be isolated, with lower isolation difficulty indicating higher anomaly likelihood. A key parameter is contamination, which allows users to specify the expected proportion of outliers in the data. This method has proven effective across various domains, including software-defined network security and industrial equipment monitoring, due to its simplicity, computational efficiency, and strong performance on complex datasets.[5]

E. General Steps for Anomaly Detection using Isolation Forest

The Isolation Forest approach for anomaly detection operates in two main phases. During the training phase, it constructs a set of isolation trees from randomly selected subsets of the input data. In the subsequent testing phase, each new data point is evaluated by traversing these trees, and an anomaly score is computed based on how easily the instance can be isolated.[7]

The process starts with data preparation, cleaning, handling missing values, and removing irrelevant features. The model is then trained by building multiple trees, where anomalies are identified as points requiring fewer splits. Predictions are made using anomaly scores, with lower scores indicating a higher abnormality. If labeled data are available, performance can be evaluated using metrics such as precision and recall. The sensitivity of the model can be adjusted by setting parameters such as the number of trees or the contamination rate before deployment. Its speed and scalability make it ideal for high-dimensional datasets.

For unlabeled data, visualization and domain expertise help validate results. The key advantage of Isolation Forest is its ability to detect anomalies without requiring extensive computational resources, making it a practical choice for real-world applications like fraud detection or system monitoring. The entire process is straightforward, balancing statistical rigor with adaptability to different use cases.

III. DATASET

The dataset, sourced from Kaggle [2], comprises real-time sensor measurements from industrial equipment deployed across multiple geographic locations.

This dataset closely resembles predictive maintenance data gathered in manufacturing plants, energy facilities, or industrial IoT environments. For example, a turbine in Atlanta might have a temperature spike or excessive vibration before failing. Using this kind of data, engineers can identify patterns that precede failures and take preventive actions.

As shown in Fig. 2, the dataset comprises 7,672 rows and 7 columns, representing sensor and contextual data collected from various types of industrial equipment operating in different geographic locations. Preliminary exploratory analysis was performed using the pandas library in Python. The data were loaded from a CSV file into a structured DataFrame, and

```

Dataset shape: (7672, 7)
Columns: ['temperature', 'pressure', 'vibration', 'humidity', 'equipment', 'location', 'faulty']
0    58.180180    25.029278    0.606516    ...    Turbine    Atlanta    0.0
1    75.740712    22.954818    2.338095    ...    Compressor    Chicago    0.0
2    71.358594    27.276830    1.389198    ...    Turbine    San Francisco    0.0
3    71.616985    32.242921    1.770690    ...    Pump    Atlanta    0.0
4    66.506832    45.197471    0.345398    ...    Pump    New York    0.0

```

Fig. 2. Dataset Overview

basic information such as the dataset’s dimensions and column names was examined. A sample of the first few entries was also reviewed to understand the format and types of values contained in the dataset.

A. Key Characteristics:

The dataset includes sensor readings such as temperature, pressure, vibration, and humidity, which are continuous variables reflecting the real-time condition of the equipment. It also contains features such as equipment, a categorical variable indicating the type of machine (e.g., Turbine, Compressor, Pump), and location, which refers to the city where the machine is deployed, helping to understand the environmental or operational context. The target label, *faulty*, is a binary indicator where 1.0 denotes a fault and 0.0 represents normal operation, serving as the basis for supervised or evaluation-based anomaly detection.

TABLE I
DATASET FEATURES AND DESCRIPTIONS

Feature	Description	Unit/Range
temperature	Equipment surface temperature	°C
pressure	Internal fluid pressure	bar
vibration	Mechanical oscillation levels	Normalized (0–1)
humidity	Ambient moisture	%
equipment	Type of machinery	Categorical
location	Deployment city	Categorical
faulty	Anomaly label	Binary (0/1)

The correlation heatmap is an essential diagnostic tool for understanding the relationships between input features. As illustrated in Figure 3, the heatmap reveals how strongly each sensor variable is linearly associated with the others. For example, a moderate positive correlation between the defective label and vibration (= 0.43) indicates that equipment exhibiting elevated vibration levels is more likely to be faulty.

This insight is valuable for two key reasons. First, it helps ensure that the anomaly detection model is not biased by highly correlated or redundant features, which can distort model performance. Second, it provides operational context, allowing engineers to identify high-risk feature interactions, such as increased vibration under certain pressure conditions so that they can proactively intervene.

In summary, the correlation heatmap serves as a foundational step in feature analysis, guiding preprocessing decisions and helping to validate the structural integrity of the dataset prior to model training.

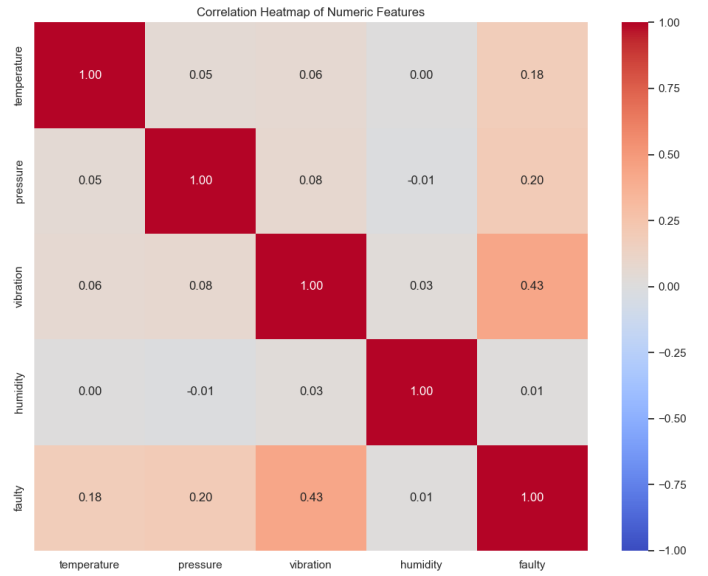


Fig. 3. Correlation heatmap

IV. METHODOLOGY

A. Libraries Used

The analysis utilizes several Python libraries for data processing and modeling. For DataFrame manipulation, we employ pandas. Numerical computations are handled using numpy. The scikit-learn library provides various tools, including StandardScaler for scaling features to zero mean and unit variance, train test split for creating a stratified 80% train 20% test split, PCA for reducing preprocessed data to two components for 2D visualization, IsolationForest as the core unsupervised anomaly detection model, and OneHotEncoder for encoding categorical features such as equipment and location. For data visualization, we use matplotlib and seaborn to create various plots including boxplots, scatterplots, heatmaps, ROC curves, PCA visualizations, pairplots, and violinplots

B. Data Preparation

The dataset consists of 7672 rows and includes numeric sensor and contextual features such as temperature, pressure, vibration, and humidity, along with a binary target variable (*faulty*), where 0 indicates normal operation and 1 denotes a fault.

Data Cleaning and Setup

The data were first reset to ensure a clean, consecutive index. The binary *faulty* column was extracted as the target variable (*y*), while all remaining numeric features were used as input features (*X*).

Exploratory Data Analysis

Initial exploratory analysis included visual inspection of summary statistics and a correlation heatmap in figure 3 , which helped to identify relationships among numeric variables and their potential association with the target label.

Train/Test Splitting

The dataset was partitioned into training (80%) and test (20%) subsets using stratified sampling to maintain the same class distribution of the `faulty` variable in both sets. The resulting test set, containing approximately 1535 samples, was held out for final model evaluation.

C. Hyperparameter Tuning

A set of hyperparameters was defined to guide the tuning of the Isolation Forest model for anomaly detection. The contamination parameter, which estimates the proportion of anomalies in the dataset, was explored over a range of values including 0.05, 0.1, and 0.15. To improve model generalization and tree diversity, the number of samples used to construct each tree (`max_samples`) was varied across 256, 512, and 'auto', the latter using the entire dataset. The number of estimators (`n_estimators`), representing the total number of trees in the ensemble, was tested at 100 and 200 to assess the trade-off between computational cost and model stability. Lastly, the maximum proportion of features considered when looking for the best split (`max_features`) was tuned using values of 0.5, 0.7, and 1.0 to evaluate the impact of feature sampling on performance.

D. Model Training and Tuning

The Isolation Forest model was initialized with a fixed random seed and configured to allow warm restarts during hyperparameter tuning. A grid search with 3-fold cross-validation was performed using `GridSearchCV` to identify the optimal combination of parameters defined earlier. The evaluation metric was the F1 score, which balances precision and recall, and the search was parallelized across available CPU cores to improve efficiency. Prior to training, the binary class labels were transformed to match the expected format of the Isolation Forest algorithm, where normal instances were mapped to +1 and faulty instances to -1. The model was then fit on the training data, and the best hyperparameters were selected based on cross-validated F1 performance..[4]

E. Visualization and Evaluation

To better understand the structure of the data and the model's performance, several visualization techniques were applied. Principal Component Analysis (PCA) was used to reduce the high-dimensional feature space to two dimensions, enabling visual inspection of normal versus faulty instances in a 2D plot. A correlation heatmap was generated to examine linear relationships among numeric features and to identify potential redundancy. Boxplots were used to compare the distributions of key features, such as temperature and vibration, across fault labels, revealing meaningful differences in central tendency. Model evaluation was further supported by plotting Receiver Operating Characteristic (ROC) curves to assess the trade-off between true positive and false positive rates, and a confusion matrix was computed to provide a detailed breakdown of classification outcomes.[4]

V. RESULTS

A. Confusionmatrix

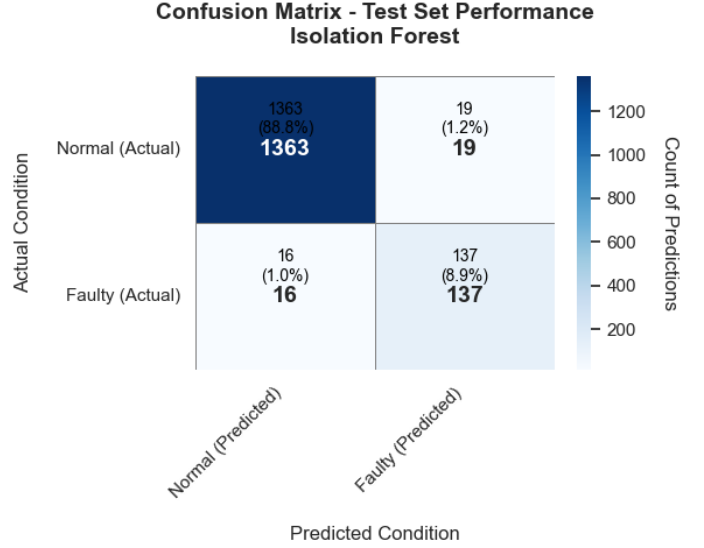


Fig. 4. Confusion Matrix Heatmap

Figure 4 presents the confusion matrix summarizing the model's performance on the labeled test set. The visualization includes both raw counts and corresponding percentages, providing a clearer breakdown of prediction outcomes. Out of all predictions, the model correctly classified 1,363 normal instances (true negatives) and 137 faulty instances (true positives). In contrast, it misclassified 19 normal instances as faults (false positives) and failed to detect 16 actual faults (false negatives).

From these results, the model achieved a precision of 0.8782, indicating that nearly 88% of instances flagged as faulty were indeed true faults. The recall was 0.8954, meaning the model successfully detected approximately 90% of all actual faults. The resulting F1-score was 0.8867, reflecting a strong balance between precision and recall. This combination of high precision and recall suggests the model is effective in both minimizing false alarms and reliably identifying faulty equipment critical traits in high-stakes industrial monitoring scenarios.

B. PCA Visualization

To visualize the distribution of predicted anomalies in the test set, Principal Component Analysis (PCA) was applied to the unscaled test data (`X_test`), projecting it into a two-dimensional space. Figure 5 displays each test instance as a point, colored according to its predicted class label—blue for normal and red for anomaly. The first two principal components explained a significant portion of the variance in the original feature space, allowing for a meaningful low-dimensional representation. The resulting scatter plot reveals that predicted anomalies tend to cluster in sparser regions of the transformed space, suggesting that the model is effectively

identifying outliers relative to the dominant data distribution. To enhance interpretability, convex hulls were overlaid to highlight the spatial extent of each class, and axis labels include the proportion of variance captured by each principal component. This visualization provides qualitative support for the model’s capacity to distinguish between normal and anomalous patterns in the test data.

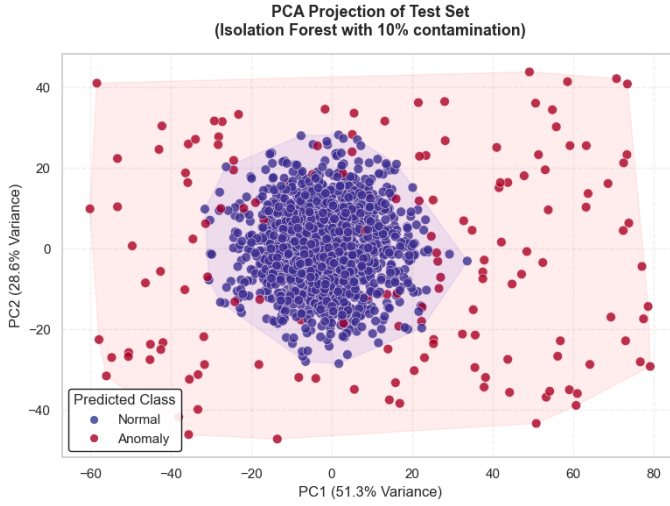


Fig. 5. PCA Visualization

C. Precision–Recall and ROC Curves

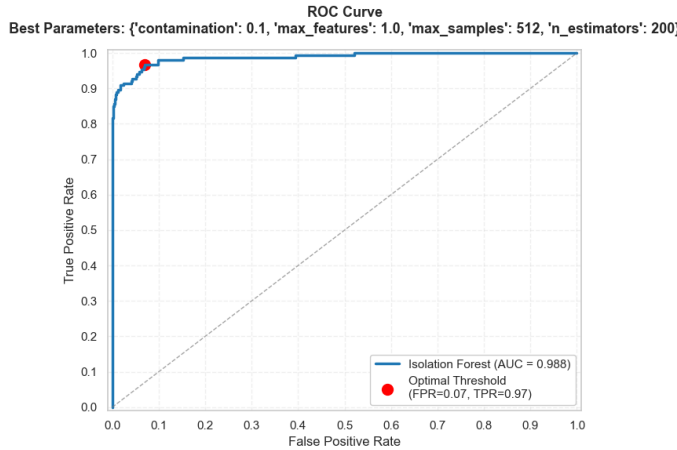


Fig. 6. ROC Curves

Figure 6 illustrates the Receiver Operating Characteristic (ROC) curve for the Isolation Forest model evaluated on the test set. The model, tuned using the best parameters from grid search (contamination set to 0.10, 512 maximum samples, 200 estimators, and all features used per split)—achieved an area under the ROC curve (AUC) of 0.988. Anomaly scores were obtained directly from the model’s decision function, and the scores were negated to align with the ROC curve’s convention of treating higher values as more likely to be positive (faulty).

The curve shows strong separability between classes, with a steep rise toward the top-left corner.

The optimal decision threshold was identified using Youden’s J statistic, which maximizes the difference between true positive rate (TPR) and false positive rate (FPR). At this threshold, the model achieved a TPR of 0.97 and an FPR of just 0.07. This point is highlighted on the curve, indicating a favorable trade-off between sensitivity and specificity. Overall, the high AUC and favorable threshold performance suggest the model is well calibrated for distinguishing faulty instances from normal behavior in industrial equipment.

VI. DISCUSSION

The results show that the Isolation Forest algorithm is well-suited for detecting equipment faults based on sensor data, achieving a strong balance between precision and recall. The model demonstrated high accuracy across both quantitative metrics (AUC = 0.988, F1 = 0.8867) and qualitative visualizations, with PCA projections confirming that many detected anomalies lie in less dense regions of the feature space.

Despite these strengths, some limitations remain. The analysis was limited to numeric features, excluding potentially informative categorical data such as equipment type and location. Additionally, the model operates on static snapshots of the data and does not account for temporal patterns, which may be important in capturing early indicators of failure.

VII. CONCLUSION AND FUTURE WORK

This study demonstrated the effectiveness of the Isolation Forest algorithm for anomaly detection in industrial equipment sensor data. Through careful hyperparameter tuning and model evaluation, the method achieved high performance across multiple metrics, including an F1-score of 0.8867 and an AUC of 0.988 on the test set. Visualizations using PCA further supported the model’s ability to distinguish anomalous patterns from normal behavior, aligning with the algorithm’s underlying assumptions.

While the results are promising, there are opportunities to improve and extend this work. Future research should explore the incorporation of categorical features and temporal dynamics, both of which may enhance fault detection accuracy. Additionally, alternative models such as Support Vector Machines (SVMs) may offer improved performance, particularly when decision boundaries between classes are more complex or when labeled data is more abundant. Comparing Isolation Forest with supervised methods like SVMs could provide valuable insights into model selection strategies for real-world deployment.

Overall, the approach presented here offers a practical, interpretable, and effective baseline for unsupervised anomaly detection in industrial monitoring systems.

REFERENCES

- [1] W. S. Al Farizi, I. Hidayah and M. N. Rizal, "Isolation Forest Based Anomaly Detection: A Systematic Literature Review," 2021 8th International Conference on Information Technology, Computer and Electrical Engineering (ICITACEE), Semarang, Indonesia, 2021, pp. 118-122, doi: 10.1109/ICITACEE53184.2021.9617498.
- [2] <https://www.kaggle.com/datasets/dnkumars/industrial-equipment-monitoring-dataset/data>
- [3] <https://www.kaggle.com/code/devraai/industrial-equipment-anomaly-detection>
- [4] <https://github.com/MoyeGI/Anomaly-Detection-Industrial-Equipment-/tree/main>
- [5] Mahmud, Md Saif Islam, Md Ashikul Rahman, Md Maruf Chakraborty, Debashon Kabir, Shaharier Shufian, Abu Sheikh, Protik. (2024). Enhancing Industrial Control System Security: An Isolation Forest-based Anomaly Detection Model for Mitigating Cyber Threats. *Journal of Engineering Research and Reports*. 26. 161-173. 10.9734/JERR/2024/v26i31102.
- [6] Liu, Fei Tony and Ting, Kai and Zhou, Zhi-Hua. (2009). Isolation Forest. 413 - 422. 10.1109/ICDM.2008.17.
- [7] F. T. Liu, K. M. Ting and Z. -H. Zhou, "Isolation Forest," 2008 Eighth IEEE International Conference on Data Mining, Pisa, Italy, 2008, pp. 413-422, doi: 10.1109/ICDM.2008.17.